

МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

**Лабораторная работа №1
по курсу “Компьютерная графика”**

Основы 3D графики

Выполнил: А. Е. Лукашов
Группа: М8О-307БВ-24
Преподаватель: В. Д. Бахарев

Москва, 2026

Условие

Цель работы:

Познакомиться с основами 3D-графики: построением простых 3D-объектов, проекцией на 2D-плоскость, а также научиться работать с матрицами перспективы, ортографической проекции и аффинными преобразованиями.

Выполненный вариант:

Вариант 4 — Усеченный правильный тетраэдр (грани — правильные треугольники и шестиугольники).

Метод решения

Геометрия:

Усеченный тетраэдр строится из правильного тетраэдра путем срезания вершин на $1/3$ и $2/3$ длины каждого ребра. В результате получается тело с: 12 вершинами, 8 гранями, 18 ребрами

Исходные вершины тетраэдра:

$$v0 = (1, 1, 1)$$

$$v1 = (1, -1, -1)$$

$$v2 = (-1, 1, -1)$$

$$v3 = (-1, -1, 1)$$

Процедурная генерация цвета: цвет каждой вершины вычисляется по формуле: $\text{color} = (\text{vertex_position} + 1.5) / 3.0$, что создает градиент на основе локальных координат.

Математика

Матрица модели:

```
model = translate(position) *  
        rotate(rotation_x, X_AXIS) *  
        rotate(rotation_y, Y_AXIS) *  
        rotate(rotation_z, Z_AXIS) *
```

```
scale(scale_factor)
```

Матрица вида:

```
view = lookAt(eye = (0, 0, 8),  
             center = (0, 0, 0),  
             up = (0, 1, 0))
```

Матрица проекции :

Перспективная:

```
proj = perspective(fov, aspect_ratio, near = 0.1, far = 100.0)
```

Ортографическая:

```
proj = ortho(-size*aspect, size*aspect, -size, size, near, far)
```

Итоговое преобразование в шейдере:

```
gl_Position = proj * view * model * vec4(position, 1.0)
```

Используемые объекты Vulkan

1. `VkBuffer` (Vertex Buffer, Index Buffer, Uniform Buffer) — хранение вершин, индексов и матриц
2. `VkPipeline` — графический конвейер с настройками растеризации, теста глубины, blending
3. `VkDescriptorSet` — передача Uniform Buffer (матрицы и цвет) в шейдеры
4. `VkRenderPass` — описание процесса отрисовки с буфером глубины
5. `VkCommandBuffer` — команды отрисовки для GPU
6. `VmaAllocator` (Vulkan Memory Allocator) — упрощенное управление памятью

Анимация траектории

Позиция фигуры вычисляется по параметрическим уравнениям:

```
position.x = radius * sin(time * speed)  
position.z = radius * cos(time * speed * 0.5)  
position.y = sin(time * speed * 1.5) * 0.5
```

`rotation.y = time * 30.0`

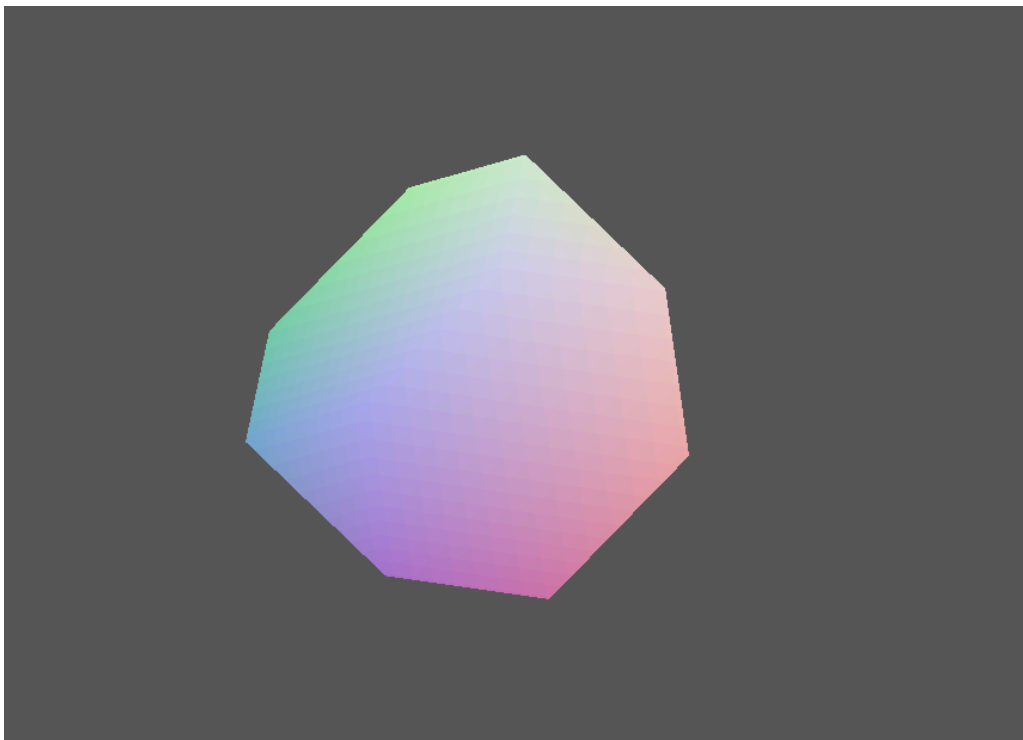
Результаты

Скриншот 1: Перспективная проекция



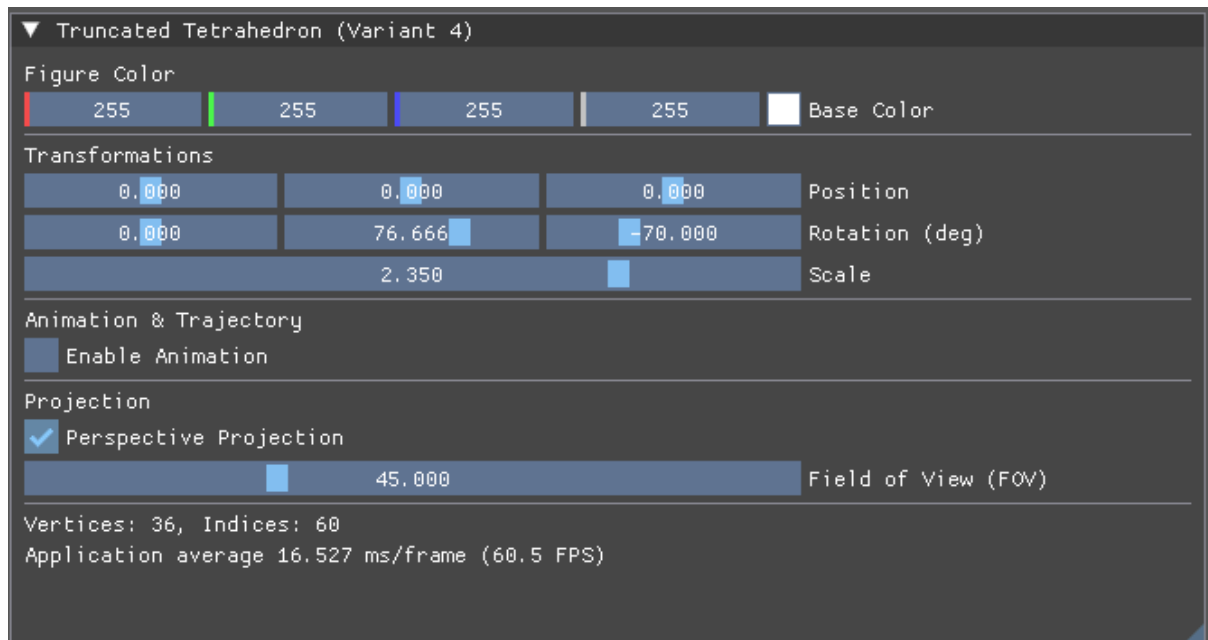
Фигура отображается с перспективной проекцией ($FOV = 45^\circ$).
Процедурная раскраска вершин создает плавные цветовые переходы.

Скриншот 2: Ортогографическая проекция



При переключении на ортографическую проекцию эффект перспективы исчезает — все грани отображаются в истинном размере независимо от расстояния до камеры.

Скриншот 3: Интерфейс управления



Интерфейс ImGUI предоставляет полный контроль над сценой

Выводы

В ходе выполнения лабораторной работы я научился:

1. Работать с Vulkan API: создавать и настраивать графический конвейер, буферы вершин и индексов, Uniform Buffer Objects для передачи данных в шейдеры, descriptor sets и pipeline layout.
2. Строить 3D-геометрию: генерировать сложные объекты (усеченный тетраэдр) программно, выполнять триангуляцию многоугольников, назначать процедурные цвета вершинам.
3. Применять матричные преобразования: использовать матрицы модели, вида и проекции для преобразования координат из локального пространства в экранное; реализовывать аффинные преобразования (перемещение, вращение, масштабирование).

4. Реализовывать разные типы проекций: переключаться между перспективной и ортографической проекцией, понимать их математические различия и области применения.
5. Создавать интерактивный интерфейс: интегрировать библиотеку ImGui с Vulkan, добавлять элементы управления для изменения параметров рендеринга в реальном времени.
6. Работать с шейдерами GLSL: писать vertex и fragment шейдеры, передавать uniform-переменные, умножать процедурные цвета вершин на базовый цвет.